

CONNECT4 C++ PROJECT STYLE GUIDE

Overview:

This style guide explains how our group will write and organize our Connect 4 program in C++. The goal is to keep the code consistent, readable, and easy to work on as a team. We want our project to clearly demonstrate object-oriented programming while also being well-structured and easy to understand.

Naming Conventions:

- We will use clear and descriptive names related to the game.
 - Classes - PascalCase
Examples: Board, Player, Game, Bot, FileLog
 - Functions and variables - camelCase
Examples: dropPiece(), checkWinner(), playerName
 - Constants - ALL_CAPS
Examples: ROWS, COLUMNS, MAX_MOVES
 - We will avoid unclear names like x, temp, or data unless used in short loops.
-

File Organization:

Our project will be split into multiple files:

- Each class will have its own .h and .cpp file
Examples: Board.h/Board.cpp, Bot.h/Bot.cpp, FileLog.h/FileLog.cpp
- main.cpp will control the overall flow (starting the game, replay loop, etc.)

This keeps the project organized and easier to debug.

Formatting:

To keep everything consistent:

- Use 4 spaces for indentation
- Opening braces stay on the same line
- Add spaces around operators

Example style:

```
If (playerWin == 0) {  
    cout << "You have lost. You'll get it next time." << endl;  
}
```

Comments:

We will only use comments when they actually help.

- Each file will start with a short description
 - Important functions like dropPiece() may include short explanations
 - Complex/important logic like checking for a win will be commented
 - We will not comment obvious lines like i++
-

Functions:

Functions will be small and focused.

Example in this project: dropPiece(int column, int playerId)

Rules:

- Each function should do one job

- Avoid putting too much logic inside main()
 - Reuse functions instead of repeating code
-

Classes:

We will use classes to organize the game.

Example:

Board Class

- Stores the game grid
 - Handles piece placement
 - Checks for wins and full board
-

Operator Overloading:

We will overload operators when it improves readability/ease.

Main use:

Overload << to print the board cleanly

Example usage: cout << board;

Composition:

We will use composition for the Bot object

Example:

The Bot class contains:

- Two board objects
- An integer array

This allows the bot to simulate potential moves in order to calculate the appropriate weights

Inheritance:

We will use inheritance for testing classes

Example :

The BotTester inherits from Bot

This allows the BotTester to access all the functions in bot not publicly accessible in order to test the full behavior of the Bot.

Exceptions and Error Handling:

We will use exceptions to handle errors.

Examples:

- Dropping a piece in a full column
- Invalid user input
- File not opening

We will:

- catch exceptions properly
 - display helpful error messages
 - prevent crashes
-

File I/O:

We will use file I/O to store game results.

Examples:

- Save winner name
- Save number of moves
- Save date/time of game

Rules:

- Always check if file opens successfully
 - Append to the file instead of overwriting
-

Loops and Conditionals:

We will use:

- for loops when we know how many times to loop
- while loops and do-while when repeating until a condition changes
- if statements for decisions

Example idea:

```
for (int j = 0; j < 7; j++) {  
    board[i][j] = 0;  
}
```

Good Programming Practices:

We will:

- keep main() simple and readable
 - avoid global variables when possible
 - initialize all variables
 - test frequently as we build
 - keep logic inside classes (not all in main)
 - write code that teammates can easily understand
-

Final Goal

Our Connect 4 project should:

- run correctly without crashing
- be organized into multiple files
- clearly demonstrate object-oriented programming
- be easy to read and follow
- show effort and good design